

Embedded Programming

Dr. Charis Kouzinopoulos

Department of Advanced Computing Sciences

Maastricht University

2024-2025

charis.kouzinopoulos@maastrichtuniversity.nl



Maastricht University

Number systems - decimal

- In elementary school, you learned to count and do arithmetic in the *decimal* system. This is called *base 10*
- Just as there are ten fingers, there are ten decimal digits, 0, 1, 2, ..., 9
- Decimal digits are joined together to form longer decimal numbers



1's column
10's column
100's column
1000's column

$$9742_{10} = 9 \times 10^3 + 7 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$$

nine seven four two
thousands hundreds tens ones

Number systems - binary

- Numbers may be represented in any base
- For example, 123 base 10 = 1111011 base 2. We can write this as $(1111011)_2$ or 0b1111011
- Numbers are kept in computer hardware as a series of high and low electronic signals, and so they are considered base 2 numbers (*remember the transistors and capacitors from the previous week?*)

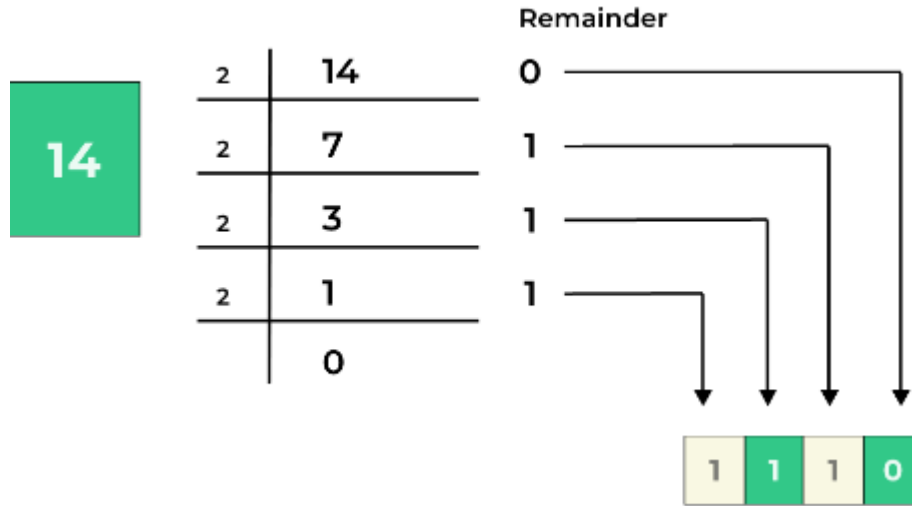
1's column	2's column	4's column	8's column	16's column	
1	1	0	1	1	

$$10110_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 22_{10}$$

one	no	one	one	no	
sixteen	eight	four	two	one	

conversion of binary to decimal

Number systems - binary



conversion of decimal to binary

Number systems - binary

- Let's work together:
- What is the binary representation of $(86)_{10}$?
- What is the decimal representation of $(10001011101)_2$?

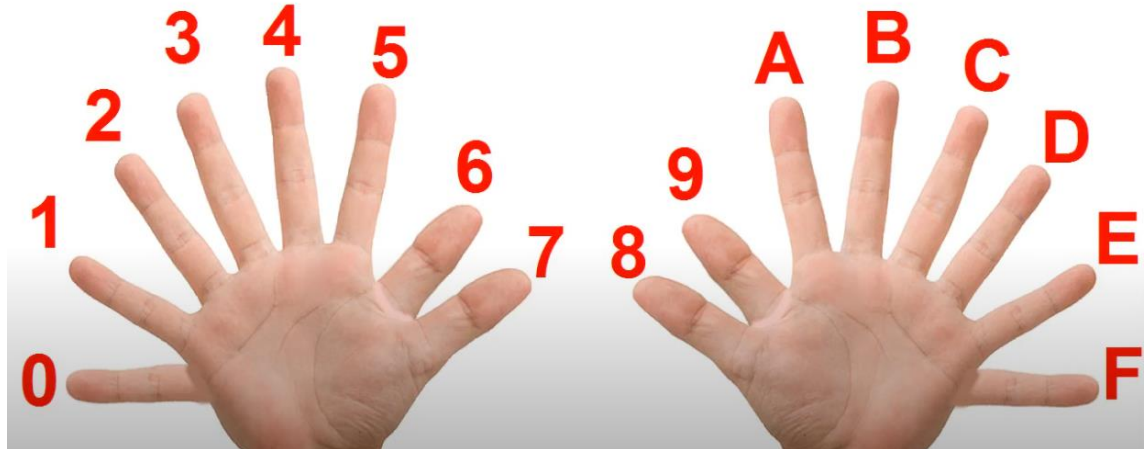
Number systems - binary

Range of 1-bit to 4-bit binary numbers

1-Bit Binary Numbers	2-Bit Binary Numbers	3-Bit Binary Numbers	4-Bit Binary Numbers	Decimal Equivalents
0	00	000	0000	0
1	01	001	0001	1
	10	010	0010	2
	11	011	0011	3
		100	0100	4
		101	0101	5
		110	0110	6
		111	0111	7
			1000	8
			1001	9
			1010	10
			1011	11
			1100	12
			1101	13
			1110	14
			1111	15

Number systems - hexadecimal

- Writing long binary numbers becomes tedious and prone to error
- A group of four bits represents one of $2^4 = 16$ possibilities. It can be more convenient to work in base 16, called hexadecimal
- Hexadecimal numbers use the digits 0 to 9 along with the letters A to F
- For example, 123 base 10 = 7B base 16. We can write this as $(7B)_{16}$ or 0x7B



Number systems - hexadecimal

- Hexadecimal numbers use the digits 0 to 9 along with the letters A to F

1's column
16's column
256's column

$$2ED_{16} = 2 \times 16^2 + E \times 16^1 + D \times 16^0 = 749_{10}$$

two hundred fifty six's fourteen sixteens thirteen ones

conversion of hexadecimal to decimal

Number systems - hexadecimal

Hexadecimal / decimal/ binary equivalents

Hexadecimal Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Representing numbers in any base

$$(\alpha_k \alpha_{k-1} \alpha_{k-2} \dots \alpha_1 \alpha_0)_b$$

$$n = \alpha_k b^k + \alpha_{k-1} b^{k-1} + \dots + \alpha_1 b^1 + \alpha_0 b^0$$

$$0 \leq a_j < b$$

***b*: base**

***a_j*: digits**

- *What is the decimal representation of $(6134)_{10}$?*
- *What is the decimal representation of $(6134)_8$?*
- *What is the decimal representation of $(6134)_7$?*
- *What is the decimal representation of $(6134)_6$?*

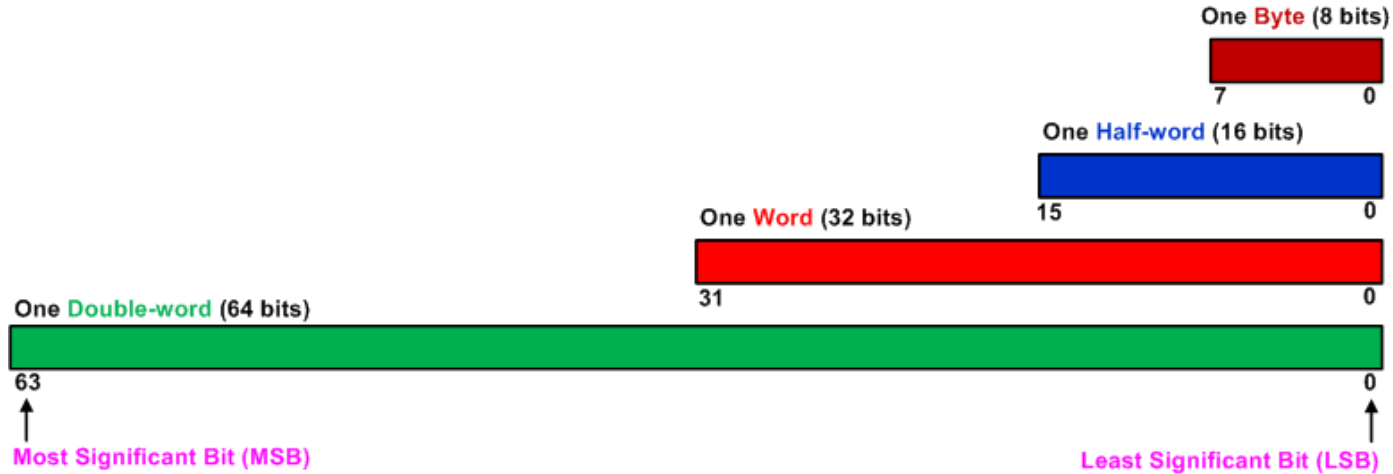
Java data types

Type	Size (in bits)	Range
byte	8	-128 to 127
short	16	-32,768 to 32,767
int	32	-2^{31} to $2^{31}-1$
long	64	-2^{63} to $2^{63}-1$
float	32	1.4e-045 to 3.4e+038
double	64	4.9e-324 to 1.8e+308
char	16	0 to 65,535
boolean	1	true or false

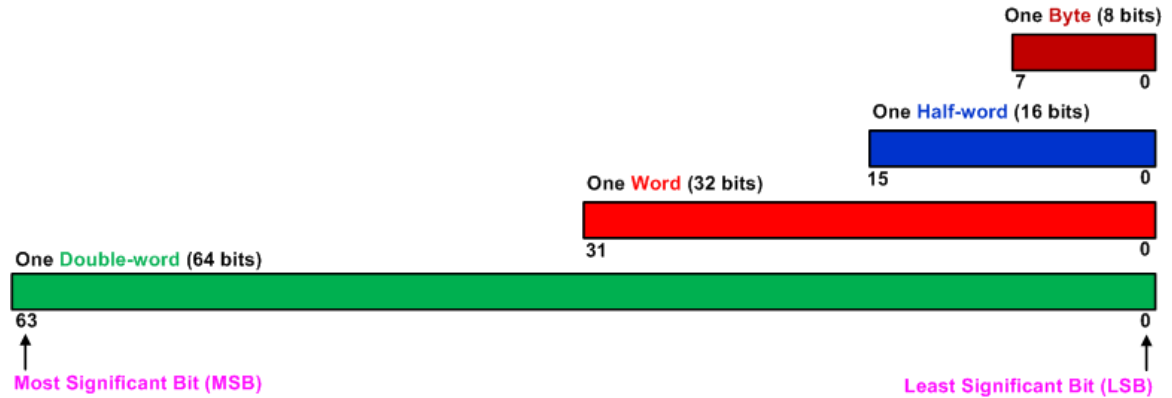
Basic Data Types in C

Data Type	Data Size (bits)	Alignment	Data Range
bool	8	byte	0 or 1. Bits 1-7 are ignored
char	8	byte	-128 - 127(signed) or 0 - 255(unsigned)
int	32	word	-2,147,483,648 - 2,147,483,647(signed) or 0 - 4,294,967,296(unsigned)
short int	16	halfword	-32,768 - 32,767(signed) or 0 - 65,536(unsigned)
long int	32	word	same as int
long long	64	word	-9,223,372,036,854,775,808 - 9,223,372,036,854,775,807(signed) or 0 - 18,446,744,073,709,551,616(unsigned)
float	32	word	+/- 1.4023x10 ⁻⁴⁵ to 3.4028x10 ⁺³⁸ , always signed
double	64	word	+/- 4.9406x10 ⁻³²⁴ to 1.7977x10 ³⁰⁸ , always signed
long double	96	word	very large range

Bit, Byte, Half-word, Word, Double-Word



Range of Unsigned Integers



Storage Size	Range	Powers of 2
Unsigned Byte	0 to 255	0 to 2^8-1
Unsigned Halfword	0 to 65,535	0 to $2^{16}-1$
Unsigned Word	0 to 4,294,967,295	0 to $2^{32}-1$
Unsigned Double-word	0 to 18,446,744,073,709,551,615	0 to $2^{64}-1$

Range of Unsigned Integers

- *unsigned int* $x = 4\ 294\ 967\ 295$;
- What is its binary representation? Hexadecimal representation?
- What will happen when $x = x + 1$; ?

Range of Unsigned Integers

- *unsigned int x = 4 294 967 295;*
- What is its binary representation? Hexadecimal representation?
- What will happen when $x = x + 1$; ?
- Hex: 0xFFFF FFFF
- Binary: 1111 1111 1111 1111 1111 1111 1111 1111

Name : x
Details: 4294967295
Default: 4294967295
Decimal: -1
Hex: 0xffffffff
Binary: 11111111111111111111111111111111
Octal: 037777777777



Name : x
Details: 0
Default: 0
Decimal: 0
Hex: 0x0
Binary: 0
Octal: 0

1010
0101 xpsr 0x61000000 (Hex)



0110

Condition Codes: APSR – Application Program Status Register

Bit	Name	Meaning after add or sub
N	negative	result is negative
Z	zero	result is zero
C	carry	signed arithmetic out of range
V	overflow	signed arithmetic out of range

- C is set upon an unsigned addition if the answer is wrong
- C is cleared upon an unsigned subtract if the answer is wrong
- V is set upon a signed addition or subtraction if the answer is wrong

Why do we care about these bits?

Range of Unsigned Integers

- *unsigned int* $x = 4\,294\,967\,295$;
- $x+=2$;
- What will be the APSR flags?

Range of Unsigned Integers

- *unsigned int x = 4 294 967 295;*
- *x+=2;*
- What will be the APSR flags?

N	Z	C	V
0	0	1	0

1010
0101 xpsr 0x21000000 (Hex)



0010

Carry and borrow

Remember:

- When adding two numbers in a column-wise manner, a **carry** occurs when the sum exceeds the number “capacity” i.e. 1 in binary or 9 in decimal. We then "carry" the extra value to the next column

$$\begin{array}{r} 1011 \\ + 1101 \\ \hline 11000 \end{array}$$

Carry and borrow

Remember:

- When subtracting two binary numbers, a **borrow** occurs when the minuend (top digit) is smaller than the subtrahend (bottom digit). In this case, we borrow from the next higher bit

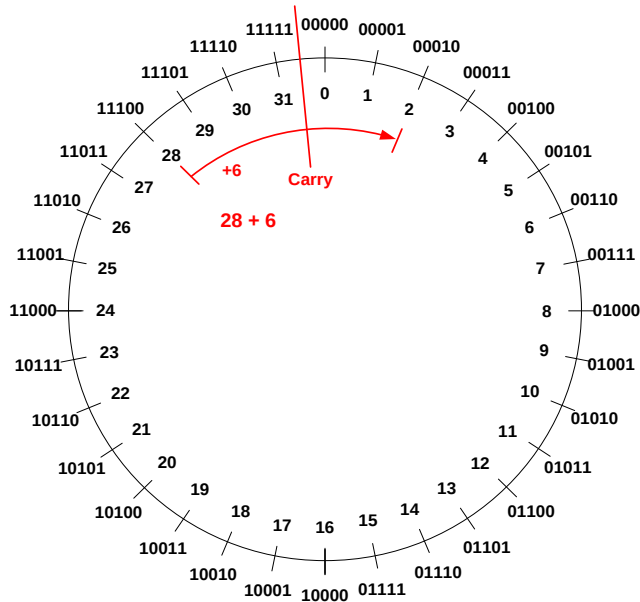
$$\begin{array}{r} 1100 \\ - 0001 \\ \hline 1011 \end{array}$$

Carry/borrow flag bit for unsigned numbers

- Given **unsigned** integers a and b
- $c = a + b$
 - Carry happens if **c is too big to fit in n bits** (i.e., $c > 2^n - 1$)
- $c = a - b$
 - Borrow happens if $c < 0$
- On ARM Cortex-M processors, the carry flag and the borrow flag are physically the same flag bit in the status register
 - **For an unsigned subtraction, Carry = NOT Borrow**

Carry/borrow flag bit for unsigned numbers

If the traverse crosses the boundary between 0 and $2^n - 1$, the carry flag is set on addition and is cleared on subtraction.



A carry occurs when adding 28 and 6

Carry	1	1	1	0	0		
	1	1	1	0	0	28	
	+	0	0	1	1	0	+ 6
	1	0	0	0	1	0	2

Extra bit is discarded.

5-bit result

- Carry flag = 1, indicating carry has occurred on unsigned addition.
- Carry flag is 1 because the result crosses the boundary between 31 and 0.

Carry/borrow flag bit for unsigned numbers

- *unsigned int a, b, c;*
- *a = 0;*
- *b = 1;*
- *c = a - b;*

- what will be the value of the c variable?
- what will be the flags of the APSR register?

Carry/borrow flag bit for unsigned numbers

- *unsigned int a, b, c;*
- *a = 0;*
- *b = 1;*
- *c = a - b;*

(x)= a	unsigned int	0
(x)= b	unsigned int	1
(x)= c	unsigned int	4294967295 (Details)

Name : c
Details:4294967295
Default:4294967295
Decimal:-1
Hex:0xfffffffff
Binary:11111111111111111111111111111111
Octal:037777777777

N	Z	C	V
1	0	0	0

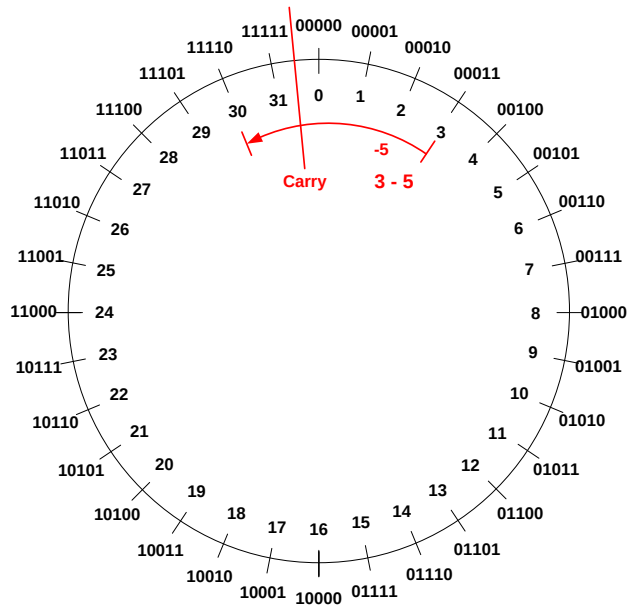
Carry/borrow flag bit for unsigned numbers

Since ARM processors do not have a dedicated borrow flag, they reuse the carry flag (C) to indicate whether borrowing is needed in subtraction:

- $C = 1$ means no borrow was needed ($R1 \geq R2$)
- $C = 0$ means a borrow was needed ($R1 < R2$)

Carry/borrow flag bit for unsigned numbers

If the traverse crosses the boundary between 0 and $2^n - 1$, the carry flag is set on addition and is cleared on subtraction.



Borrow	1	1	1	0	0
--------	---	---	---	---	---

0	0	0	1	1		3
-	0	0	1	0	1	- 5
1	1	1	1	0		30
5-bit result						

- Carry flag = 0, indicating borrow has occurred on unsigned subtraction.
- For subtraction, carry = NOT borrow.

A borrow occurs when subtracting 5 from 3.

Signed Integer Representation

5 in decimal is 0101 in binary

How to represent -5 in binary?

Signed Integer Representation

5 in decimal is 0101 in binary

How to represent -5 in binary?

Signed magnitude: $-5 = 1101$

- Can you calculate $5 + (-5)$?

Signed Integer Representation

5 in decimal is 0101 in binary

How to represent -5 in binary?

Signed magnitude: $-5 = 1101$

- Can you calculate $5 + (-5)$?
- What is the representation of $(0)_{10}$ and $(-0)_{10}$ in binary?

Signed Integer Representation

5 in decimal is 0101 in binary

How to represent -5 in binary?

Signed magnitude: $-5 = 1101$

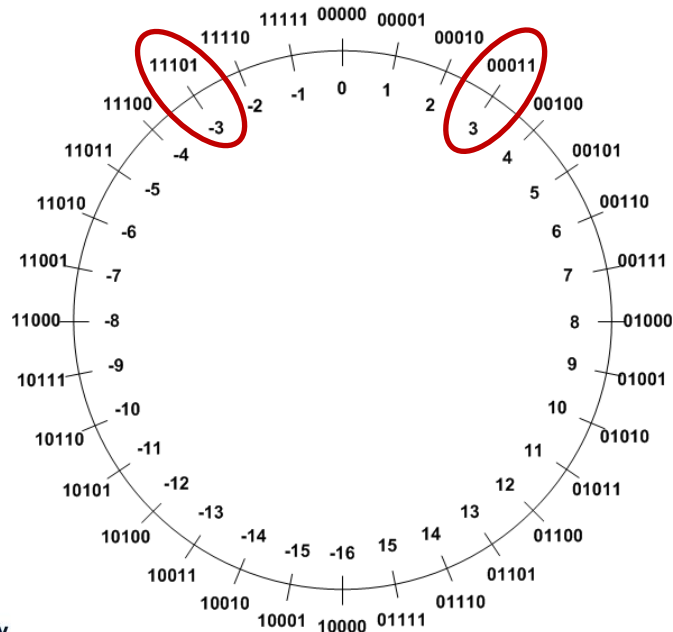
One's complement: invert (flip) all the bits in the binary representation of the number: 1010

- Can you calculate $5 + (-5)$?
- What is the representation of $(0)_{10}$ and $(-0)_{10}$ in binary?

Signed Integers Two's Complement (TC)

Two's Complement ($\bar{\alpha}$):

$$\alpha + \bar{\alpha} = 2^n$$

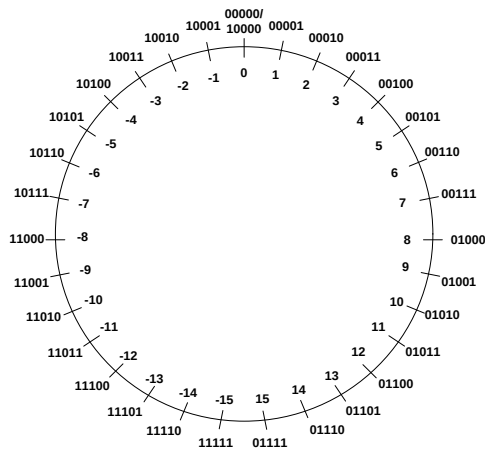


TC of a negative number can be obtained by the bitwise NOT of its positive counterpart plus one

Example 1: TC(3)

	Binary	Decimal
Original number	0b00011	3
Step 1: Invert every bit	0b11100	
Step 2: Add 1	0b11100 + 0b00001	
Two's complement	0b11101	-3

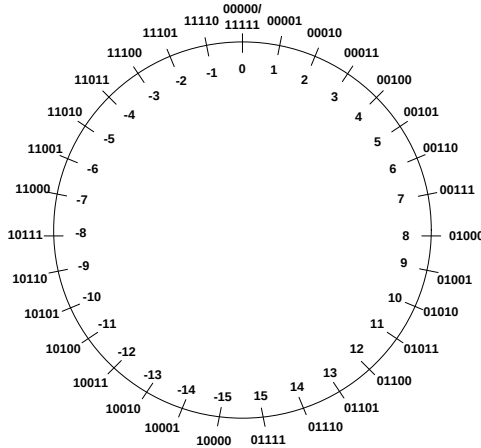
Comparison



Signed magnitude
representation

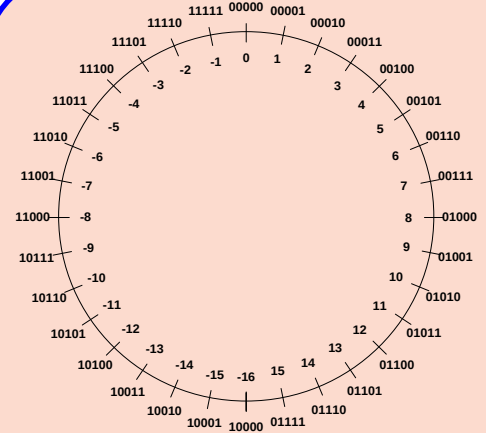
0 = positive

1 = negative



One's complement
representation

Negative = invert all
bits of a positive

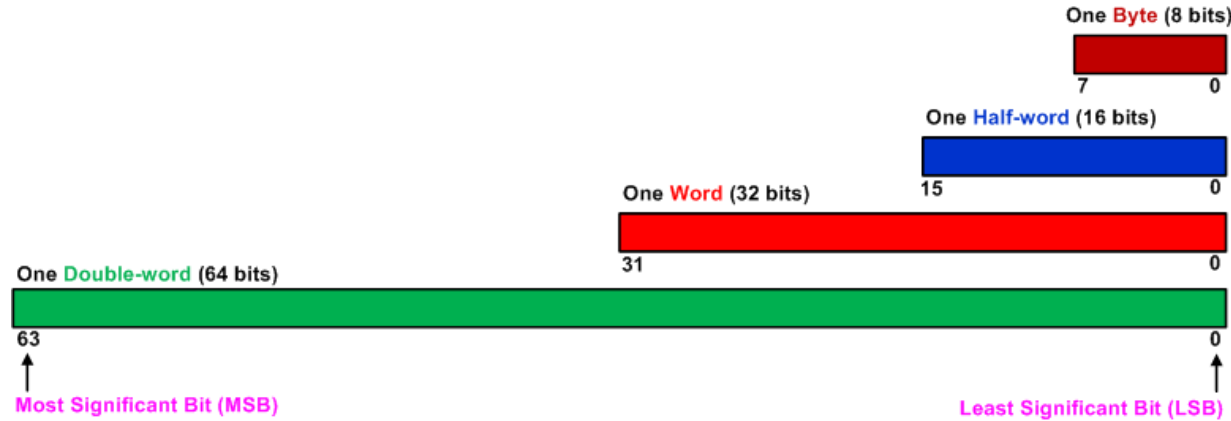


Two's Complement
representation

TC = invert all bits,
then plus 1

Used in modern computers!

Range of Signed Integers (TC)



Storage Size	Range	Powers of 2
Signed Byte	-128 to +127	-2^7 to 2^7-1
Signed Halfword	-32,768 to +32,767	-2^{15} to $2^{15}-1$
Signed Word	-2,147,483,648 to +2,147,483,647	-2^{31} to $2^{31}-1$
Signed Double-word	-9,223,372,036,854,775,808 to +9,223,372,036,854,775,807	-2^{63} to $2^{63}-1$

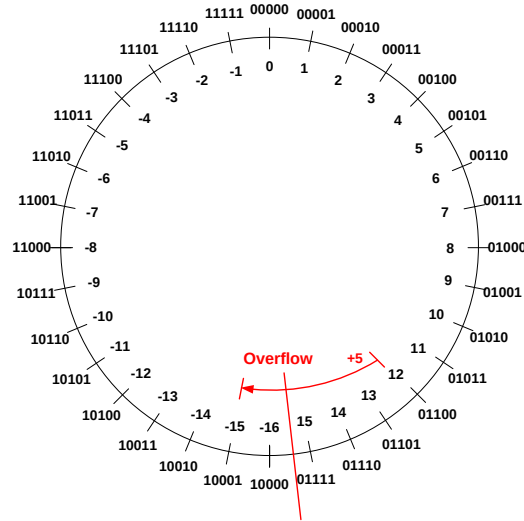
Overflow for Signed Add/Sub

- **Overflow** occurs in **signed** binary arithmetic when the result of an operation is too large (or too small) to be represented in the given number of bits
- When adding signed numbers represented in two's complement, overflow occurs only in two scenarios:
 - **Adding** two **positive** numbers produces a **negative** result
 - **Adding** two **negative** numbers produces a **positive** result
- Similarly, when subtracting signed numbers, overflow occurs in two scenarios:
 - **Subtracting** a **positive** number from a **negative** number but getting a **positive** result
 - **Subtracting** a **negative** number from a **positive** number but producing a **negative** result
- Overflow cannot occur when adding operands with different signs or when subtracting operands with the same signs

Overflow for Signed Add/Sub

- **Overflow** occurs in **signed** binary arithmetic when the result of an operation is too large (or too small) to be represented in the given number of bits
- When adding signed numbers represented in two's complement, overflow occurs only in two scenarios:
 - $+A + +B = -C$
 - $-A + -B = +C$
- Similarly, when subtracting signed numbers, overflow occurs in two scenarios:
 - $+A - -B = -C$
 - $-A - +B = +C$
- Overflow cannot occur when adding operands with different signs or when subtracting operands with the same signs

Overflow for Signed Add

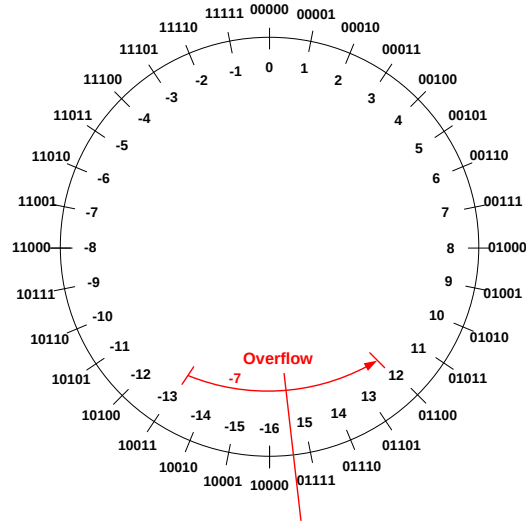


Overflow occurs when adding two positive integers but getting a negative result.

0	1	1	0	0	12		
+	0	0	1	0	1	+	5
1	0	0	0	1		-15	
5-bit result							

1. On addition, overflow occurs if $sum \geq 2^4$ when adding two positives.
2. Overflow never occurs when adding two numbers with different signs.

Overflow for Signed Add



Overflow occurs when adding two negative integers but getting a positive result.

	1 0 0 1 1	-13
+	1 1 0 0 1	+ -7
	1 0 1 1 0 0	12

Extra bit is discarded.

5-bit result

On addition, overflow occurs if $sum < -2^4$ when adding two negatives.

All in one slide!

- **Carry:**
 - C is set upon an unsigned addition if the answer is wrong
 - C is cleared upon an unsigned subtract if the answer is wrong
- **Overflow:**
 - $+A + +B = -C$
 - $-A + -B = +C$
 - $+A - -B = -C$
 - $-A - +B = +C$

Signed or Unsigned?

$a = 0b10000000$ (128 OR -128)

$b = 0b10000000$ (128 OR -128)

$c = a + b$

- ▶ Are a and b signed or unsigned numbers?
 - CPU does not know the answer at all
 - Therefore, the hardware sets up both the carry flag and the overflow flag
 - It is the software's (programmer's/compiler's) responsibility to interpret the flags

1010 0101 xpsr	0x21000000 (Hex)
-------------------	------------------



0010

Signed or Unsigned?

$a = 0b10000000$

$b = 0b10000000$

$c = a + b$

- ▶ Are a and b signed or unsigned numbers?

```
uint a;  
uint b;  
...  
c = a + b  
...
```

Check the carry flag!

C Program

Signed or Unsigned?

$a = 0b10000000$

$b = 0b10000000$

$c = a + b$

- ▶ Are a and b signed or unsigned numbers?

```
int a;  
int b;  
...  
c = a + b  
...
```

C Program

Check the overflow flag!

Bitwise operations

Bitwise operations

A	B	A B	A & B	A ^ B
0	0	0	0	0
1	0	1	0	1
0	1	1	0	1
1	1	1	1	0

- AND (&)
- OR (|)
- XOR (^)
- NOT (!)

Bitwise operations - AND

unsigned int **A** = 0x3B //00111011;

unsigned int **B** = 0x96 //10010110;

	00111011	3B
&	10010110	96
	00010010	12

Bitwise operations - OR

unsigned int **A** = 0x3B //00111011;

unsigned int **B** = 0x96 //10010110;

	00111011	3B
	10010110	96
	10111111	BF

Bitwise operations - XOR

unsigned int **A** = 0x3B //00111011;

unsigned int **B** = 0x96 //10010110;

	00111011	3B
$\wedge$	10010110	96
	10101101	AD

Bitwise operations - Shift

```
unsigned int A = 0x3B    //00111011;
```

```
C = A << 2; // left shift A by 2
```

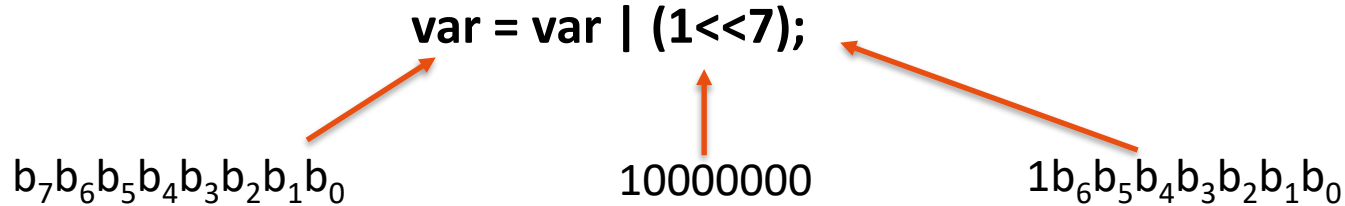
```
C = 11101100 //EC
```

```
C = A >> 4;
```

```
C = 00000011 //3
```


Bitwise operations – Setting bits

Let's say we have a variable `var` and we want to set bit 7 without modifying the other bits:

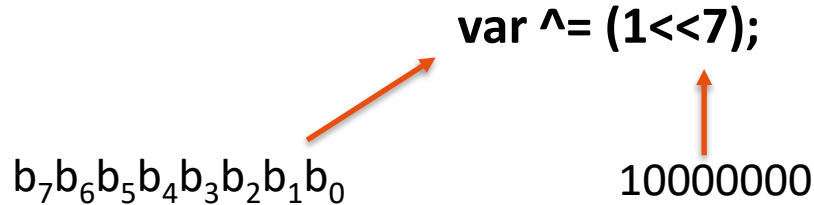


(we usually write this in a compact form):

`var |= (1<<7);`

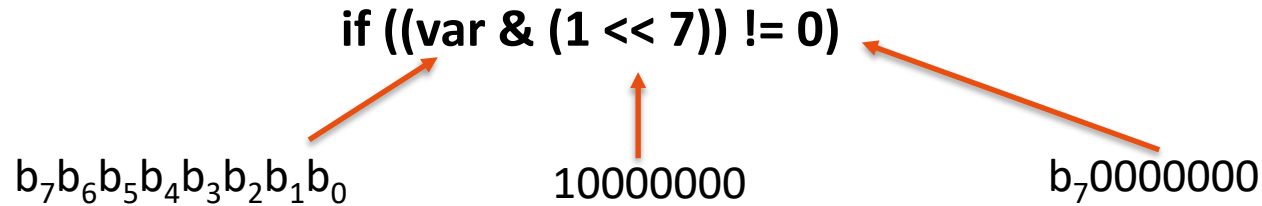
Bitwise operations – Toggling (inverting) bits

Let's say we have a variable `var` and we want to invert bit 7 without modifying the other bits:



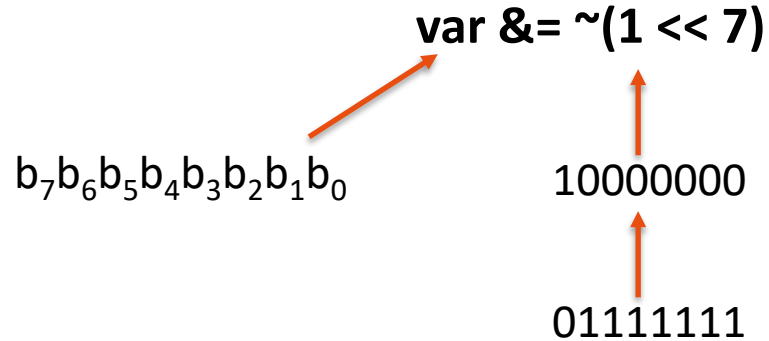
Bitwise operations – Testing bits

Let's say we have a variable `var` and we want to test if bit 7 is set:



Bitwise operations – Clearing bits

Let's say we have a variable `var` and we want to clear bit 7:



Bitwise operations - exercises

unsigned int **A** = 0x3B //00111011;

How to set the 7th bit? What will be the result?

In the resulting number, how to flip (invert) the 5th bit?

Bitwise operations - exercises

unsigned int **A** = 0x3B //00111011;

How to set the 7th bit? What will be the result?

$B = B \mid (1 \ll 7)$

$B \mid= (1 \ll 7)$

B = 10111011 //0xBB

In the resulting number, how to flip (invert) the 5th bit?

$B \wedge= (1 \ll 5)$

B = 10011011 //0x9B

Bitwise operations - exercises

unsigned int **A** = 0x3B //00111011;

How to test if the 4th bit is 0 or 1?

How to clear the 3rd bit?

Bitwise operations - exercises

```
unsigned int A = 0x3B    //00111011;
```

How to test if the 4th bit is 0 or 1?

if (A & (1 << 4) /* check to see if bit 4 is set */

How to clear the 3rd bit?

A &= ~(1 << 3) ;

Bitwise operations

unsigned int **A** = 0x3B //00111011;

How to retrieve the value of the 5th bit?

Bitwise operations - exercises

unsigned int **A** = 0x3B //00111011;

How to retrieve the value of the 5th bit?

(A >> 5) & 1;

Takeaways

- Number systems: decimal, binary, hexadecimal
- Representing numbers in any base
- Bit, Byte, Half-word, Word, Double-Word
- Signed and unsigned representation
- Two's complement
- Condition codes and the APSR register
- Bitwise operations: AND, OR, XOR, NOT